Preface: The Gap Between Discrete and Continuous

Prefer to read this offline? Download the complete, formatting-optimized Vector Embeddings Ebook here.

The preceding series on Tokenization established the mathematical process by which raw text is compressed into a sequence of discrete integer IDs. The subsequent series on Transformers begins with a dense, continuous tensor of embedding vectors already loaded with semantic meaning. Between these two endpoints exists a gap: the mechanism by which discrete integers become rich, continuous vectors whose geometric relationships encode the distributional structure of language.

This series derives that mechanism from first principles.

The Scope

The derivation proceeds through a shallow, two-layer neural network whose sole purpose is to train the embedding matrix W_E. Every mathematical operation is computed explicitly using a concrete toy model:

- **Vocabulary ($V = 8$):** `The` `quick` `brown` `fox` `jumps` `over` `lazy` `dog`
- **Embedding Dimension (d_{model}):** 3 dimensions.

This toy model is intentionally minimal. Eight words and three dimensions are sufficient to demonstrate every operation on a whiteboard, yet complex enough to exhibit the geometric phenomena (orthogonality, convergence, linear substructures) that define trained embedding spaces at scale.

The Architecture

The series follows the complete lifecycle of a single training step, then traces the cumulative effect of billions of such steps:

1. **One-Hot Encoding and the Embedding Matrix.** The discrete token ID is converted into a sparse one-hot vector and projected through W_E to extract a dense embedding.
2. **The Continuous Vector Space.** The geometric properties of the randomly initialized embedding space (isotropy, concentration of measure, and mutual orthogonality) are derived.
3. **The Forward Pass, Softmax, and Cross-Entropy Loss.** The embedding is projected through an un-embedding matrix W_U to produce logits, converted to probabilities via Softmax, and evaluated against the ground truth via Cross-Entropy Loss.
4. **Backpropagation Through Loss and Softmax.** The gradient signal is traced backwards through the loss and Softmax layers, deriving the unified $\hat{\mathbf{y}} - \mathbf{y}$ expression from first principles.
5. **Backpropagation Through the Weight Matrices.** The gradient propagates through W_U and W_E, demonstrating how one-hot sparsity ensures only a single embedding row is updated per training example.
6. **Convergent Geometry and Linear Substructures.** The cumulative effect of billions of gradient updates is traced, showing how distributional statistics organically produce semantic clustering and vector arithmetic.
7. **The Embedding Tensor and the Limits of Static Representations.** The single-token lookup is generalized to full-sequence processing. The fundamental limitations of static embeddings (context-blindness and order-agnosticism) are identified, establishing the handoff to the Transformer architecture.

Every partial derivative is derived explicitly. Every abstract result is grounded in the toy vocabulary. The mathematics speaks for itself.

Prefer to read this offline? Download the complete, formatting-optimized Vector Embeddings Ebook here.

Chapter 1: One-Hot Encoding and the Embedding Matrix

The preceding series on Tokenization established the rigorous mathematical process of transforming variable, unstructured text into discrete integer sequences. Through subword compression algorithms like Byte Pair Encoding, it was demonstrated how a system can organically parse language into a highly optimized, finite vocabulary.

When text passes through this tokenizer, a word like "walking" might be decomposed into two distinct subword tokens, each mapped to a specific integer ID in our vocabulary:

`walk` (ID: 4) `ing` (ID: 12)

This is where the Tokenization pipeline ends. However, handing the integer IDs `4` and `12` directly to a neural network presents an immediate mathematical dead end.

The Problem with Discrete Integers

Deep learning models operate through continuous mathematics: matrix multiplication, calculus, and gradient descent. Discrete integer IDs are merely categorical labels; they possess no inherent algebraic meaning.

If the IDs `4` and `12` are fed directly into a neural network, the model will attempt to perform mathematical operations on them. It might deduce that `12` is three times as large as `4`. It might conclude that the "distance" between `walk` (4) and `ing` (12) is exactly 8. This is mathematically nonsensical. The integer ID `4` does not mean the concept of walking is "smaller" than the suffix `ing`. They are just arbitrary index numbers.

To participate in a neural network, these discrete, categorical IDs must be transformed into continuous, dense vectors where every dimension represents a tunable parameter.

The One-Hot Encoding

The first step in this transformation is to convert the categorical ID into a mathematically neutral format: a one-hot encoded vector.

If the total vocabulary size (V) is 9, the token `walk` (ID 4) can be represented as a vector of length 9 containing all zeros, except for a single `1` at the 4th index.

$$\mathbf{x}_{walk} = [0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]_{1\times 9}$$

This sparse vector allows the network to process the categorical identity of the token without accidentally inferring any false mathematical magnitude. The `1` simply states "this token is present", while the `0`s state "these other tokens are not". However, a one-hot vector is completely sparse and contains no semantic depth. It must be projected into a continuous space.

The Embedding Matrix (W_E)

To give these tokens continuous algebraic meaning, a massive grid of random numbers known as the embedding matrix is initialized, typically denoted as W_E .

This matrix serves as a dense lookup table. It contains exactly one row for every possible token in the architecture's predefined vocabulary, and the width of every row is defined by the network's internal model dimensionality (d_{model}).

If the vocabulary size V is 9, and the vector dimensionality d_{model} is 512, the embedding matrix requires exactly 9×512 parameters. At the moment of initialization, every single one of these parameters is just a random decimal number.

The Mathematical Extraction

How does the network transition from the sparse one-hot vector to the dense 512-dimensional vector? Through standard matrix multiplication.

By multiplying the one-hot vector $\mathbf{x}_{walk}$ by the embedding matrix W_E , the mathematics dictate that all rows multiplied by `0` are canceled out, leaving only the 4th row (multiplied by `1`) to pass through.

$$\mathbf{x}_{walk} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{1\times 9} \times \begin{bmatrix} w_{1,1} & \cdots & w_{1,512} \\ w_{2,1} & \cdots & w_{2,512} \\ w_{3,1} & \cdots & w_{3,512} \\ w_{4,1} & \cdots & w_{4,512} \\ \vdots & \ddots & \vdots \\ w_{9,1} & \cdots & w_{9,512} \end{bmatrix}_{9\times 512}$$

$$= \begin{bmatrix} w_{4,1} & w_{4,2} & \cdots & w_{4,512} \end{bmatrix}_{1\times 512}$$

This matrix multiplication involves no learning; it is the linear algebra mechanism for selecting a specific row from a table. The token `walk` has now been successfully mapped from a discrete integer ID into a continuous, 512-dimensional vector of random numbers.

The gap into the continuous vector space has been bridged. The nature of these random numbers, the structure of high-dimensional geometry, and the mathematical implications of randomly scattering tokens across this landscape are explored next.

Chapter 2: The Continuous Vector Space

The linear projection of a one-hot vector through the embedding matrix W_E fundamentally alters the mathematical domain of the tokens. The gap from a discrete, categorical space ($\mathbb{Z}$) into a dense, continuous vector space ($\mathbb{R}^{d_{model}}$) has been bridged.

This transition is a structural prerequisite for deep learning.

The Differentiable Manifold

Neural networks learn through calculus—specifically, by calculating gradients and traversing a loss landscape via backpropagation. Calculus requires a continuous, differentiable manifold. The gradient of a discrete integer ID cannot be calculated, nor can a microscopic mathematical adjustment be made to a categorical label.

By projecting tokens into a dense, continuous space, each token becomes parameterized by a vector of floating-point numbers. If the architecture dictates a dimension size of $d_{model} = 512$, the token `walk` is now defined by 512 independent, tunable parameters:

$$\mathbf{v}_{walk} = [0.124, -0.841, 0.339, \dots, -0.052] \in \mathbb{R}^{512}$$

Because these parameters are continuous, they are entirely unconstrained. A gradient descent step can apply an arbitrarily small perturbation to $\mathbf{v}_{walk}$ (e.g., subtracting 0.001 from its first dimension) to incrementally improve the model's objective function. This continuous adjustment mechanism is what allows the network to gradually map semantic relationships into geometric proximity.

The Isotropic Expanse

These initial 512 dimensions do not correspond to latent human concepts. At initialization, one dimension does not encode "plurality," while another encodes "sentiment."

When a model is instantiated, the weights of W_E are populated by sampling from a random probability distribution, such as a standard normal distribution $\mathcal{N}(0, \sigma^2)$. Because every dimension is drawn independently from the exact same symmetric distribution, the resulting vector space is **isotropic**—meaning it looks completely uniform in every direction.

High-dimensional geometry invalidates the intuition of a 3D cloud of points with varied clustering. In 512 dimensions, a mathematical phenomenon called the *concentration of measure* dominates. Calculating the length of a vector requires summing its 512 squared dimensions. Due to the massive sample size, the Law of Large Numbers dictates that the final lengths of all the randomized vectors will average out to be virtually identical. Instead of a solid, uneven blob of points, the random initialization creates a perfectly uniform, hollow shell. Every single token sits on the exact same surface of a high-dimensional hypersphere, devoid of any structural bias.

Consider the vastness of a high-dimensional space. In a standard 2D plane, there are exactly 4 geometric quadrants. Randomly scattering a vocabulary of 50,000 tokens across a 2D plane inevitably creates dense clusters due to spatial constraints.

However, a 512-dimensional space contains 2^{512} distinct quadrants (mathematically known as orthants). To put that scale into perspective, 2^{512} is approximately 1.3×10^{154} . When a token is initialized, it is dropped onto the hypersphere in one of these 1.3×10^{154} orthants. The mathematical surface area dictates that every single token in the vocabulary lands in profound isolation. Accidental clustering is mathematically improbable.

The Geometry of Orthogonality

Within this vast, isolated expanse, a geometric phenomenon occurs: random initial vectors for `walk` and `ing` are mathematically guaranteed to be virtually **orthogonal** (perpendicular) to one another.

The relationship between two vectors is measured to understand this. In the context of neural networks, **cosine similarity** is predominantly used.

The geometric formula for the dot product is $A \cdot B = ||A|| \times ||B|| \times \cos(\theta)$. By dividing the dot product of two vectors by their lengths ($||A|| \times ||B||$), $\cos(\theta)$ is isolated, yielding the cosine similarity. This formula reveals that the cosine of the angle between two vectors is directly proportional to their **dot product**—the mathematical operation of pairing up corresponding dimensions, multiplying them together, and summing the results.

When two dimensions randomly drawn from a distribution centered at zero are multiplied, the product is equally likely to be positive (if the signs agree) or negative (if the signs disagree).

In a simple 3D space, the dot product only sums 3 of these randomized terms. Due to the small sample size, variance dominates. It is highly probable that all 3 terms will randomly agree, resulting in a large positive sum. Consequently, in low dimensions, random vectors frequently point in roughly the same direction.

But in a 512-dimensional space, the dot product sums 512 independent terms. At this massive scale, variance shrinks and the Law of Large Numbers strictly takes over. Probability dictates that the outcomes must forcefully regress to the mean. An almost perfect balance is mathematically guaranteed: roughly 256 products will be positive, and the exact other half will be negative.

Furthermore, because the underlying probability distribution is symmetrical, there is no mathematical bias skewing the absolute size of these numbers. The 256 positive terms are, on average, exactly as large as the 256 negative terms.

When summed together, the positive and negative halves perfectly cancel each other out, collapsing the entire dot product to exactly zero. Because the cosine of the angle is proportional to this dot product, a value of zero results in a cosine of 0.0 . In trigonometry, the angle whose cosine is exactly zero is a 90-degree angle. Therefore, any two random tokens—whether they are `walk` and `ing` , or `walk` and `dog` —begin their existence geometrically orthogonal to one another.

A Feature, Not a Bug

This natural propensity toward orthogonality is a structurally advantageous property.

Orthogonality provides a mathematically clean, unentangled starting point. If the vectors started out randomly clustered, the network would have to spend significant computational effort separating those accidental correlations. Because every token begins maximally uncorrelated and non-committal, a true blank slate is presented. The architecture possesses the maximal bandwidth to selectively pull related tokens together and push unrelated tokens apart, driven purely by the objective truth of the training data.

The embedding matrix begins as an orthogonal coordinate system. The tokens possess no semantic clustering and no grammatical hierarchy. The imposition of structural meaning upon this isotropic expanse—by evaluating predictions and continuously adjusting these free parameters—is the core mechanism of end-to-end learning explored next.

Chapter 3: The Forward Pass, Softmax, and Cross-Entropy Loss

To transform a randomly initialized embedding matrix (W_E) into a mathematically rigorous semantic space, a mechanism must be built to evaluate its current state. This is achieved by constructing a simple neural network *around* W_E . Its sole objective is **Next-Token Prediction**: observing a sequence of tokens and predicting the most probable subsequent token in the corpus.

The Proxy of Prediction

The choice of next-token prediction requires justification. The ultimate goal is to capture the semantic meaning of a word, but there is no database of "semantic features" (e.g., `is_animal` = `1.0`) to supervise the network.

Instead, the system relies on **Distributional Semantics**, famously summarized by linguist J.R. Firth: *"You shall know a word by the company it keeps."* If the network observes that the words `fox` , `wolf` , and `dog` are all frequently followed by words like `hunts` , `sleeps` , or `runs` , the calculus of the network is mathematically forced to push their vectors closer together in the geometric space. Semantic meaning is not explicitly programmed; it is an *emergent byproduct* of the next-token prediction objective.

The 2-Layer Embedding Network

When training static embeddings from scratch, the architecture is astonishingly simple. It is a shallow neural network consisting of exactly two layers:

1. **Layer 1: The Embedding Projection (W_E).** The W_E matrix *is* the first layer of the network. It projects the one-hot encoded vocabulary vector (which represents the current context word) into the dense, continuous vector space (dimension d_{model}) to extract its currently learned semantic meaning.
2. **Layer 2: The Un-embedding Projection (W_U).** The W_U matrix has a shape of $d_{model} \times V$. Just like W_E , its internal values are completely random at initialization and are learned during training. This layer projects the dense continuous vector (which now contains the contextual semantic features extracted by Layer 1) back out into the vocabulary space (dimension V), producing a raw score for every possible word in the vocabulary. This score represents the unnormalized likelihood that each specific word is the correct next token in the sequence.

The Hidden Layer

In this architecture, the **hidden layer** is simply the state of the network between these two projections. It is the dense d_{model} -dimensional vector itself. The network's only true purpose is to optimize the weights of Layer 1 (W_E). Once training is complete, Layer 2 (W_U) is entirely discarded, and the hidden layer outputs (the vectors) are saved as the final embeddings.

The Activation Function

Crucially, **there are no non-linear activation functions** (like ReLU or GELU) between Layer 1 and Layer 2.

This is the fundamental architectural difference between an Embedding Network and the Feed-Forward Networks (FFN) found in Transformers. A Transformer uses non-linear activations to learn complex, non-linear logic. The Embedding Network intentionally omits them because its goal is to construct a pure, linear geometric space (where concepts like cosine similarity are mathematically sound).

If two linear matrices (W_E and W_U) are stacked without a non-linearity, they mathematically collapse into a single operation ($W_E \times W_U = W_{combined}$). The *only* reason they are kept separated is because the data must be intercepted in the middle—at the hidden layer—to extract the embeddings.

A Toy Mathematical Walkthrough

The following toy model demonstrates the forward pass through explicit calculation:

- **Vocabulary ($V = 8$):** `The` `quick` `brown` `fox` `jumps` `over` `lazy` `dog`
- **Embedding Dimension (d_{model}):** 3 dimensions.

The sequence is `The` `quick` `brown`. The network's task is to predict the next token: `fox`.

(Note: If the sequence is at the end of a sentence, the target is the End of Sequence `<EOS>` token. `<EOS>` is treated exactly like any other word; it receives a row in W_E and its vector's geometry is learned purely through observing sentence-ending patterns.)

Step 1: The Input

The final word of the context, `brown`, is represented as a one-hot vector. The word `brown` is at index 2 in the vocabulary.

$$\mathbf{x} = [0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]$$

Step 2: Layer 1 (The Embedding Lookup)

The one-hot vector $\mathbf{x}$ is multiplied by the randomly initialized 8×3 embedding matrix, W_E.

$$W_E = \begin{bmatrix} 0.1 & -0.4 & 0.2 \\ 0.5 & 0.1 & -0.8 \\ \mathbf{-0.3} & \mathbf{0.9} & \mathbf{0.4} \\ 0.2 & -0.2 & 0.1 \\ \dots & \dots & \dots \end{bmatrix}$$

Because all values in $\mathbf{x}$ are 0 except at index 2, the dot product $\mathbf{x} \times W_E$ simply extracts row 2.

$$\mathbf{h} = [-0.3 \quad 0.9 \quad 0.4]$$

This 3-dimensional vector is the **hidden layer** state. It is the current, untrained embedding for `brown`.

Step 3: Layer 2 (The Un-embedding Projection)

This hidden state is then projected back into the 8-word vocabulary space using the un-embedding matrix, W_U (a 3×8 matrix).

$$\mathbf{z} = \mathbf{h} \times W_U$$

Assume the resulting vector $\mathbf{z}$ evaluates to:

$$\mathbf{z} = [1.2 \quad -0.5 \quad 0.3 \quad 2.1 \quad -1.8 \quad 0.7 \quad 0.0 \quad 0.4]$$

This vector $\mathbf{z}$ contains the **logits**. These raw, unbounded geometric scores indicate how strongly the network believes each vocabulary word is the next token.

Step 4: The Softmax Function

To evaluate the network's performance, these arbitrary logits must be converted into a valid probability distribution where all values are strictly positive and sum to 1.0. This is achieved mathematically via the **Softmax** function:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

The Softmax for index 3 (`fox`) is calculated as follows:

1. **Exponentiate z_3:**

$$e^{2.1} \approx 8.16$$

2. **Exponentiate all logits and sum them:**

$$e^{1.2} + e^{-0.5} + e^{0.3} + e^{2.1} + e^{-1.8} + e^{0.7} + e^{0.0} + e^{0.4} \approx 3.32 + 0.60 + 1.35 + 8.16 + 0.16 + 2.01 + 1.0 + 1.49 = 18.09$$

3. **Divide:**

$$\frac{8.16}{18.09} \approx 0.45$$

The network predicts a 45% probability that `fox` is the next word. Applying this operation across all 8 indices yields our predicted probability distribution, $\hat{\mathbf{y}}$.

Step 5: The Cross-Entropy Loss

Because the embeddings are randomly initialized, a 45% prediction is the result of chance. To mathematically force the network to learn, the exact divergence of this prediction from reality must be quantified.

The ground truth, $\mathbf{y}$, is that `fox` (index 3) is definitely the next word. The true probability distribution is a one-hot vector where index 3 is 1.0, and all other indices are 0.

The **Cross-Entropy Loss** calculates the divergence between the predicted distribution ($\hat{\mathbf{y}}$) and the true distribution ($\mathbf{y}$).

The choice of this specific operation stems from information theory, where its purpose is to mathematically measure "surprise." The mechanics of the equation break into two parts:

1. **The Logarithm as Surprise**: Taking the logarithm of the predicted probability ($\log(\hat{y}_i)$) converts a raw percentage into a continuous measurement of surprise. If the network predicts a 99% probability for a word, its surprise is near zero ($\log(0.99) \approx -0.01$). If it predicts a 1% probability, its surprise is massive ($\log(0.01) \approx -4.6$).
2. **The Summation as Expected Value**: In statistics, the "expected value" of a system is calculated by taking every possible outcome, multiplying it by its true probability, and summing them together. By multiplying the true probability (y_i) by the network's surprise ($\log(\hat{y}_i)$) and summing across the entire vocabulary, the equation calculates the *expected surprise* of the network.

By computing this expected surprise across the entire vocabulary space, the total mathematical distance between the network's holistic belief and reality is measured:

$$L = -\sum_{i=1}^{V} y_i \log(\hat{y}_i)$$

Because the true distribution $\mathbf{y}$ is one-hot, a simplification occurs. For every incorrect word, the true probability y_i is 0. This causes every single term in the summation to mathematically vanish, except for the one correct word where $y_i = 1.0$.

Consequently, the massive summation radically collapses into a single term evaluating only the correct answer. This reduced form is known as **Negative Log-Likelihood**:

$$L = -\log(\hat{y}_{true})$$

The conceptual interpretation is straightforward:

- **Likelihood**: This is the model's predicted probability for the correct word (in this case, 0.45 for `fox`). The objective is to maximize this value.
- **Log**: Taking the natural logarithm of a decimal between 0 and 1 yields a negative number (e.g., $\log(0.45) \approx -0.79$).
- **Negative**: Multiplication by -1 flips this into a positive "loss" scalar. Higher confidence yields lower loss; lower confidence yields higher loss.

Plugging in our prediction for `fox`:

$$L = -\log(0.45) \approx 0.79$$

This scalar $L = 0.79$ represents the absolute error of the forward pass. If the network had predicted a 99% probability ($-\log(0.99)$), the loss would be exponentially near zero. If it had predicted 1% ($-\log(0.01)$), the loss would be massive (4.6).

The Limit of Static Embeddings: Superposition

The fundamental limitation of this architecture is **polysemy** (words with multiple meanings).

Because W_E is a static matrix, there is exactly one row allocated for the word `apple`. During training, sentences like *"I ate an apple"* will mathematically pull this vector toward the fruit cluster. Conversely, sentences like *"Apple released a phone"* will pull the exact same vector toward the tech cluster.

Over billions of words, the single vector for `apple` settles into a state of **superposition**—the mathematical center of mass between its usages. It becomes a noisy, blurred average.

This static limitation is the exact reason the **Self-Attention** mechanism was invented. In a Transformer, Self-Attention allows the static `apple` vector to look at the surrounding words in the sequence and dynamically morph itself purely into a tech vector or purely into a fruit vector before the prediction is made.

The physical adjustment of the vector space via gradient descent, which produces these static baseline vectors in the first place, is the subject of the next chapter.

Chapter 4: Backpropagation Through Loss and Softmax

The previous chapter ended with a single scalar: $L = 0.79$. This number is the network's total error—a mathematical verdict on the quality of the forward pass. To learn, the network must translate this single number into specific, targeted corrections to thousands of individual weights distributed across two matrices (W_U and W_E).

The mechanism that accomplishes this is **Backpropagation**: the systematic application of the chain rule from calculus, working backwards from the loss through each layer of the network. The output is the **gradient**—a vector of partial derivatives that dictates how much each weight contributed to the error, and in which direction it must move to reduce it.

From Scalar to Signal

The expression $\frac{\partial L}{\partial w}$ asks a precise question: *if this single weight w is nudged by an infinitesimally small amount, while holding every other weight in the network frozen, how does the loss L change?*

The goal is to drive the loss to zero—a perfect prediction. If the derivative is positive, increasing w increases the loss. If it is negative, increasing w decreases the loss. When the derivative is exactly zero, the weight is sitting at a point where the loss is locally minimized. The magnitude dictates how *sensitive* the loss is to this particular weight: a large magnitude means small changes to w cause large swings in the loss, while a magnitude near zero means the weight is nearly irrelevant to the prediction.

Computing this derivative for a weight in the final layer is straightforward, as it directly connects to the loss. For a weight in W_E , located at the first layer, the relationship is indirect: a change in W_E alters the hidden state $\mathbf{h}$, which alters the logits $\mathbf{z}$, which alters the Softmax probabilities $\hat{\mathbf{y}}$, which finally alters the loss L . The **chain rule** decomposes this dependency into a product of simple, local derivatives:

$$\frac{\partial L}{\partial W_E} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial W_E}$$

Each factor in this product is a tractable computation. Each derivative is now derived, starting from the loss and working backwards to W_E .

The Loss Derivative: $\frac{\partial L}{\partial \hat{\mathbf{y}}}$

The first link in the chain is straightforward. The question is: how does the loss change when the predicted probabilities change?

Recall from Chapter 3 that the Cross-Entropy Loss, after collapsing under the one-hot encoded truth vector, reduces to:

$$L = -\log(\hat{y}_{true})$$

The loss depends on exactly one element of the predicted distribution: $\hat{y}_{true}$, the probability the network assigned to the correct word (`fox` , at index 3). It is completely independent of the probabilities assigned to every other word. This means the partial derivative of the loss with respect to any incorrect word's probability is simply zero:

$$\frac{\partial L}{\partial \hat{y}_k} = 0 \quad \text{for all } k \neq true$$

For the correct word, the derivative of $-\log(x)$ with respect to x is required. This is a standard result from calculus: the derivative of $\log(x)$ is $\frac{1}{x}$, and the leading negative sign carries through:

$$\frac{\partial L}{\partial \hat{y}_{true}} = \frac{-1}{\hat{y}_{true}}$$

This result has a clear interpretation. If the network assigned a high probability to the correct word (say, $\hat{y}_{true} = 0.95$), the derivative is small: $\frac{-1}{0.95} \approx -1.05$. The loss is barely sensitive to further changes—the network is already nearly correct. But if the network assigned a tiny probability (say, $\hat{y}_{true} = 0.01$), the derivative is massive: $\frac{-1}{0.01} = -100$. The loss is *extremely* sensitive, generating a strong signal to fix the prediction. The negative sign indicates that *increasing* $\hat{y}_{true}$ will *decrease* the loss.

The Softmax Derivative: $\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}}$

The second link in the chain is more involved. The question becomes: how does each Softmax output $\hat{y}_i$ change when a single logit z_j is adjusted?

The three index variables used throughout this derivation are defined as follows:

- i — the index of the word whose **probability** is being measured (the "target" of the observation).
- j — the index of the word whose **logit** is being altered (the "cause" of the change).
- k — a summation placeholder that ranges over all V words in the vocabulary. It appears only inside $\sum_k$ expressions.

Recall the Softmax definition:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{k=1}^{V} e^{z_k}}$$

This is a fraction: a numerator (e^{z_i}) divided by a denominator ($\sum_k e^{z_k}$). The denominator is a sum over *all* logits in the vocabulary. Changing a single logit z_j does not only affect the probability $\hat{y}_j$ for that word—it also changes the denominator, which in turn changes *every other* probability $\hat{y}_i$.

To differentiate this fraction, the **quotient rule** from calculus is applied. For any function expressed as a fraction $\frac{f}{g}$, the quotient rule states:

$$\frac{d}{dx}\left(\frac{f}{g}\right) = \frac{f' \cdot g \; - \; f \cdot g'}{g^2}$$

where f' denotes the derivative of the numerator and g' denotes the derivative of the denominator.

Here, the Softmax numerator is $f = e^{z_i}$ and the denominator is $g = \sum_k e^{z_k}$. Applying the quotient rule requires f' and g' —the derivatives of the numerator and denominator with respect to the logit z_j being differentiated. The derivative of the numerator e^{z_i} depends entirely on whether z_j is the *same* variable as z_i or a *different* one.

If measuring how the probability of `fox` changes when adjusting the logit for `fox` itself ($i = j$), the numerator $e^{z_{fox}}$ depends on the variable of differentiation, making its derivative non-zero. If measuring how the probability of `fox` changes when adjusting the logit for `The` ($i \neq j$), the numerator $e^{z_{fox}}$ does not contain z_{The} at all—it is a constant, and its derivative is zero. This fundamental difference in the numerator's derivative produces two structurally different results, requiring the two cases to be handled separately.

Case 1: $i = j$ (same word)

The probability $\hat{y}_i$ is differentiated with respect to its *own* logit, z_i . This models how the predicted probability of `fox` changes when the logit for `fox` is adjusted.

The derivatives of both the numerator and denominator with respect to z_i are calculated as follows:

- **Numerator derivative**: $f' = \frac{\partial}{\partial z_i} e^{z_i} = e^{z_i}$. The exponential function is its own derivative, and since the numerator *does* contain z_i , the result is non-zero.
- **Denominator derivative**: $g' = \frac{\partial}{\partial z_i} \sum_k e^{z_k} = e^{z_i}$. The sum contains many terms ($e^{z_1}, e^{z_2}, \ldots$), but only the one term e^{z_i} depends on z_i . All other terms are constants with respect to z_i and vanish under differentiation.

Substituting f , g , f' , and g' into the quotient rule formula $\frac{f'g - fg'}{g^2}$:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i} \cdot \sum_k e^{z_k} \; - \; e^{z_i} \cdot e^{z_i}}{\left(\sum_k e^{z_k}\right)^2}$$

Factoring e^{z_i} out of the numerator yields:

$$= \frac{e^{z_i} \left(\sum_k e^{z_k} \; - \; e^{z_i}\right)}{\left(\sum_k e^{z_k}\right)^2}$$

This single fraction is then split into a product of two fractions:

$$= \frac{e^{z_i}}{\sum_k e^{z_k}} \cdot \frac{\sum_k e^{z_k} \; - \; e^{z_i}}{\sum_k e^{z_k}}$$

The first fraction is, by definition, $\hat{y}_i$. The second fraction is $1 - \hat{y}_i$ (the full sum minus the i -th term, divided by the full sum). Therefore:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$$

This result has a natural interpretation. The derivative is largest when $\hat{y}_i \approx 0.5$ —maximum uncertainty—and shrinks toward zero as $\hat{y}_i$ approaches 0 or 1 . A confident prediction is *insensitive* to small logit perturbations; an uncertain one is highly responsive. This is a fundamental property of sigmoid-family functions, reflecting the fact that Softmax **saturates** at its extremes.

Case 2: $i \neq j$ (different words)

The probability $\hat{y}_i$ is differentiated with respect to a *different* logit, z_j . This models how the predicted probability of `The` changes when the logit for `fox` is adjusted.

The numerator and denominator derivatives with respect to z_j are:

- **Numerator derivative**: $f' = \frac{\partial}{\partial z_j} e^{z_i} = 0$. The numerator e^{z_i} does not contain z_j at all—it is a constant with respect to z_j —so its derivative is zero. This is the key difference from Case 1.
- **Denominator derivative**: $g' = \frac{\partial}{\partial z_j} \sum_k e^{z_k} = e^{z_j}$. Just as before, only the one term e^{z_j} in the sum depends on z_j .

Substituting into the quotient rule formula $\frac{f'g - fg'}{g^2}$:

$$\frac{\partial \hat{y}_i}{\partial z_j} = \frac{0 \cdot \sum_k e^{z_k} \; - \; e^{z_i} \cdot e^{z_j}}{\left(\sum_k e^{z_k}\right)^2}$$

The first term in the numerator vanishes (anything multiplied by zero is zero), leaving:

$$= \frac{-e^{z_i} \cdot e^{z_j}}{\left(\sum_k e^{z_k}\right)^2}$$

This splits into a product of two fractions, each recognized as a Softmax output:

$$= -\frac{e^{z_i}}{\sum_k e^{z_k}} \cdot \frac{e^{z_j}}{\sum_k e^{z_k}} = -\hat{y}_i \cdot \hat{y}_j$$

Recall from Chapter 3 that all Softmax outputs must sum to exactly 1.0. The total probability is fixed. If the logit for `fox` (z_j) increases, the probability of `fox` ($\hat{y}_j$) grows. To maintain a sum of 1.0, every other word's probability must shrink.

The derivative $-\hat{y}_i \cdot \hat{y}_j$ dictates exactly how much each word loses. For example, if the probability of `fox` is $\hat{y}_{fox} = 0.45$ and the probability of `The` is $\hat{y}_{The} = 0.18$, then the rate at which `The` loses probability when the logit for `fox` is increased is $-0.18 \times 0.45 = -0.081$. If the probability of `jumps` is only $\hat{y}_{jumps} = 0.009$, it loses probability at a rate of $-0.009 \times 0.45 = -0.004$. Words holding larger probability mass have more to lose; words near zero are barely affected.

Summary of the Softmax Derivative

These two cases fully describe how the Softmax function responds to logit changes. Case 1 dictates how a word's own probability responds to its own logit, while Case 2 dictates how every other word's probability responds. These results are now combined with the loss derivative to compute the gradient of the loss with respect to the logits, $\frac{\partial L}{\partial \mathbf{z}}$.

Combining: The Gradient at the Logits ($\frac{\partial L}{\partial \mathbf{z}}$)

The first two links of the chain have been computed independently. The loss derivative (Step 1) dictates how the loss responds to changes in the predicted probabilities. The Softmax derivative (Step 2) dictates how the predicted probabilities respond to changes in the logits. These are combined via the chain rule to determine how the loss responds to changes in the logits:

$$\frac{\partial L}{\partial z_i} = \sum_{k=1}^{V} \frac{\partial L}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_i}$$

This summation ranges over every word in the vocabulary, because changing a single logit z_i affects every Softmax output $\hat{y}_k$. However, Step 1 established that $\frac{\partial L}{\partial \hat{y}_k} = 0$ for every $k \neq true$. This means every term in the sum where k is an incorrect word multiplies by zero and vanishes. Only the term where $k = true$ survives:

$$\frac{\partial L}{\partial z_i} = \frac{-1}{\hat{y}_{true}} \cdot \frac{\partial \hat{y}_{true}}{\partial z_i}$$

The appropriate Softmax derivative case is substituted depending on whether z_i is the logit for the correct word or an incorrect word.

For the correct class ($i = true$): Case 1 is used, where $\frac{\partial \hat{y}_{true}}{\partial z_{true}} = \hat{y}_{true}(1 - \hat{y}_{true})$:

$$\frac{\partial L}{\partial z_{true}} = \frac{-1}{\hat{y}_{true}} \cdot \hat{y}_{true}(1 - \hat{y}_{true})$$

The $\hat{y}_{true}$ in the denominator cancels with the $\hat{y}_{true}$ in the numerator:

$$= -(1 - \hat{y}_{true}) = \hat{y}_{true} - 1$$

For every incorrect class ($i \neq true$): Case 2 is used, where $\frac{\partial \hat{y}_{true}}{\partial z_i} = -\hat{y}_{true} \cdot \hat{y}_i$:

$$\frac{\partial L}{\partial z_i} = \frac{-1}{\hat{y}_{true}} \cdot (-\hat{y}_{true} \cdot \hat{y}_i)$$

The two negative signs cancel each other, and $\hat{y}_{true}$ cancels between the numerator and denominator:

$$= \hat{y}_i$$

The Unified Gradient Expression

The two results are summarized as follows:

- **For the correct class** ($i = true$, which is `fox` at index 3): the gradient is $\hat{y}_{true} - 1$.
- **For every incorrect class** ($i \neq true$, such as `The` , `quick` , `brown` , etc.): the gradient is $\hat{y}_i$.

These appear as two different formulas. However, the true label vector $\mathbf{y}$ is one-hot encoded—it contains 1 at index 3 (`fox`) and 0 at every other index. Writing each case as $\hat{y}_i - y_i$ yields the following:

- For `fox` (index 3): $y_3 = 1$, so $\hat{y}_3 - y_3 = \hat{y}_{true} - 1$. This matches our correct-class result.
- For `The` (index 0): $y_0 = 0$, so $\hat{y}_0 - y_0 = \hat{y}_0 - 0 = \hat{y}_0$. This matches our incorrect-class result.
- For `quick` (index 1): $y_1 = 0$, so $\hat{y}_1 - y_1 = \hat{y}_1$. Same pattern.

Both cases follow the identical formula: $\hat{y}_i - y_i$. The one-hot structure of $\mathbf{y}$ naturally absorbs the case distinction—the -1 only appears at the correct class because that is the only index where y_i is non-zero. The gradient for *every* logit in the vocabulary is therefore expressed as a single, unified vector expression:

$$\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}$$

The entire pipeline—quotient rules, logarithmic derivatives, case-splitting across the Softmax—reduces entirely to **predicted minus truth**.

Cross-Entropy Loss is mathematically structured to pair with Softmax and produce this result. The logarithm in the loss algebraically cancels with the exponential in Softmax, and the normalization structure ensures the gradient is always bounded and well-behaved. This coupling is why Cross-Entropy is the standard loss function for classification: it guarantees numerically stable gradients pointing directly toward the correct answer.

For our toy network, this gradient is interpretable at a glance. Every incorrect word in the vocabulary receives a *positive* gradient (its predicted probability minus zero), signaling that its logit should decrease. The correct word `fox` receives a *negative* gradient ($\hat{y}_{fox} - 1$, a number less than zero), signaling that its logit should increase. The magnitudes are self-calibrating—the most overconfident wrong answers receive the strongest corrections.

The gradient at the logits has been established. The next chapter continues the backward pass through the weight matrices.

Chapter 5: Backpropagation Through the Weight Matrices

The previous chapter established the gradient at the logits: $\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}$. The backward pass now continues through the weight matrices to compute the gradient at the embedding layer itself.

Propagating Through Layer 2: $\frac{\partial \mathbf{z}}{\partial \mathbf{h}}$

The first two links of the chain have been combined to form $\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}$. The next link is $\frac{\partial \mathbf{z}}{\partial \mathbf{h}}$: how do the logits change when the hidden state (the embedding vector for `brown`) is altered?

Recall from Chapter 3 that the logits are computed by multiplying the hidden state $\mathbf{h}$ by the un-embedding matrix W_U:

$$\mathbf{z} = \mathbf{h} \times W_U$$

The result $\mathbf{z}$ is a vector with 8 elements—one logit per vocabulary word (`The`, `quick`, `brown`, `fox`, `jumps`, `over`, `lazy`, `dog`). In our toy model, $\mathbf{h}$ is a 1×3 vector (the 3-dimensional embedding for `brown`) and W_U is a 3×8 matrix. Each logit z_j is therefore the dot product of $\mathbf{h}$ with column j of W_U.

To differentiate this, the operations are expanded element-wise. The index m refers to a specific **embedding dimension** (ranging from 1 to 3 in this model). Written out for a single logit z_j:

$$z_j = h_1 \cdot W_{U_{1,j}} + h_2 \cdot W_{U_{2,j}} + h_3 \cdot W_{U_{3,j}}$$

This is a sum of three terms, each multiplying one embedding dimension h_m by its corresponding weight $W_{U_{m,j}}$. Differentiating this sum with respect to a specific embedding dimension h_m models how z_j changes when h_m is adjusted:

The weights $W_{U_{1,j}}$, $W_{U_{2,j}}$, and $W_{U_{3,j}}$ are constants—they are fixed parameters of the network during this step. So each term in the sum has the form "constant $\times$ variable" or "constant $\times$ unrelated variable." Taking the derivative with respect to h_m:

- The term $h_m \cdot W_{U_{m,j}}$ contains h_m, and the derivative of a variable multiplied by a constant is simply the constant: $W_{U_{m,j}}$.
- Every other term (e.g., $h_1 \cdot W_{U_{1,j}}$ when $m \neq 1$) does not contain h_m at all. These are constants with respect to h_m, and their derivatives are zero.

The entire sum collapses to a single surviving term:

$$\frac{\partial z_j}{\partial h_m} = W_{U_{m,j}}$$

The derivative is simply the weight that multiplied h_m in the forward pass. This makes intuitive sense: the weight $W_{U_{m,j}}$ controls exactly how much influence the m-th embedding dimension has on the j-th logit. A large weight means that dimension has a strong effect on that logit; a weight near zero means it has almost none.

The Gradient at the Hidden State ($\frac{\partial L}{\partial \mathbf{h}}$)

This result is chained with $\frac{\partial L}{\partial \mathbf{z}}$ to yield the gradient of the loss with respect to $\mathbf{h}$. A single embedding dimension h_m contributes to *every* logit in $\mathbf{z}$ (it participates in all 8 dot products). This requires a sum over all 8 vocabulary words:

$$\frac{\partial L}{\partial h_m} = \sum_{j=1}^{8} \frac{\partial L}{\partial z_j} \cdot \frac{\partial z_j}{\partial h_m} = \sum_{j=1}^{8} (\hat{y}_j - y_j) \cdot W_{U_{m,j}}$$

Each term in this sum isolates how much embedding dimension m of `brown`'s vector contributed to the error at vocabulary word j. The error at each word ($\hat{y}_j - y_j$) is multiplied by the weight connecting h_m to that word ($W_{U_{m,j}}$), and summed across all 8 words.

Computing this for all three embedding dimensions ($m = 1, 2, 3$) simultaneously yields the complete gradient in matrix notation:

$$\frac{\partial L}{\partial \mathbf{h}} = (\hat{\mathbf{y}} - \mathbf{y}) \cdot W_U^T$$

The transpose W_U^T reverses the direction of the projection. During the forward pass, W_U projected the 3 -dimensional embedding $\mathbf{h}$ *up* into 8 -dimensional vocabulary space to produce the logits. Now, W_U^T projects the 8 -dimensional error signal *back down* into 3 -dimensional embedding space. The result is a 1×3 gradient vector that lives in the same coordinate system as `brown`'s embedding vector—it tells us the exact direction and magnitude that `brown`'s vector should be nudged to reduce the loss.

Updating W_U Along the Way

Before continuing backward, note that W_U itself contains weights that require updating. Their gradients are derived from the same element-wise equation:

$$z_j = h_1 \cdot W_{U_{1,j}} + h_2 \cdot W_{U_{2,j}} + h_3 \cdot W_{U_{3,j}}$$

This time, the expression is differentiated with respect to a specific *weight* $W_{U_{m,j}}$ instead of an embedding dimension h_m. The roles flip: the embedding values h_1, h_2, h_3 are constants (computed during the forward pass and fixed), and $W_{U_{m,j}}$ is the variable.

- The term $h_m \cdot W_{U_{m,j}}$ contains our variable, and the derivative of a constant multiplied by a variable is the constant: h_m.
- Every other term (e.g., $h_1 \cdot W_{U_{1,j}}$ when $m \neq 1$) does not contain $W_{U_{m,j}}$, so its derivative is zero.

Therefore:

$$\frac{\partial z_j}{\partial W_{U_{m,j}}} = h_m$$

Applying the chain rule to get the gradient of the loss:

$$\frac{\partial L}{\partial W_{U_{m,j}}} = \frac{\partial L}{\partial z_j} \cdot \frac{\partial z_j}{\partial W_{U_{m,j}}} = (\hat{y}_j - y_j) \cdot h_m$$

Each weight's gradient is the product of two values: the error signal at the vocabulary word it connects to ($\hat{y}_j - y_j$), and the embedding dimension value that flowed through it during the forward pass (h_m). These updates are applied alongside the W_E updates at the end.

Propagating Through Layer 1: $\frac{\partial \mathbf{h}}{\partial W_E}$

The backward pass now reaches the first layer—the origin of the forward pass and the last link in the chain. Recall:

$$\mathbf{h} = \mathbf{x} \times W_E$$

where $\mathbf{x}$ is the one-hot input vector for `brown`. This is another matrix multiplication, identical in structure to the one just differentiated. The W_E matrix has 8 rows (one per vocabulary word) and 3 columns (one per embedding dimension). Written element-wise for a single embedding dimension h_m:

$$h_m = x_1 \cdot W_{E_{1,m}} + x_2 \cdot W_{E_{2,m}} + x_3 \cdot W_{E_{3,m}} + \cdots + x_8 \cdot W_{E_{8,m}}$$

Each term multiplies the input value at a vocabulary index (x_1 for `The`, x_2 for `quick`, x_3 for `brown`, etc.) by the corresponding weight in row i of W_E. Just as in Layer 2, the input values x_i are constants (fixed from the one-hot encoding) and $W_{E_{i,m}}$ is the variable. Only the term containing $W_{E_{i,m}}$ survives differentiation:

$$\frac{\partial h_m}{\partial W_{E_{i,m}}} = x_i$$

The chain rule is applied to connect this to the loss. Because $\frac{\partial L}{\partial h_m}$ was computed in the previous section:

$$\frac{\partial L}{\partial W_{E_{i,m}}} = \frac{\partial L}{\partial h_m} \cdot \frac{\partial h_m}{\partial W_{E_{i,m}}} = \frac{\partial L}{\partial h_m} \cdot x_i$$

But $\mathbf{x}$ is one-hot: $x_i = 0$ for every vocabulary word except `brown` (index 2), where $x_2 = 1$. This means:

- For the row of `The` ($i = 0$): $\frac{\partial L}{\partial W_{E_{0,m}}} = 0 \cdot \frac{\partial L}{\partial h_m} = 0$. The gradient is zero. That row is untouched.
- For the row of `quick` ($i = 1$): $\frac{\partial L}{\partial W_{E_{1,m}}} = 0 \cdot \frac{\partial L}{\partial h_m} = 0$. Same—zero gradient.
- For the row of `brown` ($i = 2$): $\frac{\partial L}{\partial W_{E_{2,m}}} = 1 \cdot \frac{\partial L}{\partial h_m} = \frac{\partial L}{\partial h_m}$. The gradient passes through directly.
- For every remaining row (`fox`, `jumps`, `over`, `lazy`, `dog`): zero gradient.

This is the identical "plucking" mechanism observed in the forward pass, operating in reverse. During the forward pass, the one-hot vector *extracted* a single row from W_E. During backpropagation, it *deposits* the gradient into that same single row. Only `brown`'s vector receives an update from this training example. Every other word in the vocabulary remains untouched.

The Weight Update

With the gradient computed end-to-end, the fundamental weight update rule is applied:

$$W_{new} = W_{old} - \alpha \cdot \nabla L$$

The scalar α is the **learning rate**, and its role is critical. Why not apply the full gradient and move directly to the position that would perfectly predict `fox`? Because `brown` does not exist in a single context. Across the training corpus, `brown` precedes `fox`, but also `bear`, `sugar`, `eyes`, and `shoes`. Each of these contexts generates a different gradient, pulling `brown`'s vector in a different direction in the embedding space.

If the learning rate is too large, each training example violently overwrites the previous one. The vector for `brown` oscillates chaotically, never settling into a meaningful position. A small learning rate ensures each individual example contributes only a microscopic nudge. Over millions of iterations, these nudges accumulate statistically: directions that are consistently reinforced across many contexts grow dominant, while contradictory or noisy signals cancel out. The final resting position of the vector reflects not any single sentence, but the **aggregate statistical geometry** of every context in which `brown` appeared.

After this single training step, the 3-dimensional vector for `brown` has shifted by a fraction of a decimal point. The change is imperceptible. But the same operation is about to execute billions of times, across every token in the corpus, adjusting every vector that participates in each forward pass. The cumulative effect of these microscopic nudges—and the geometric structure that organically emerges from them—is the subject of the next chapter.

Chapter 6: Convergent Geometry and Linear Substructures

After a single training step, the vector for `brown` has shifted by a fraction of a decimal point. One gradient, computed from one sentence, adjusts three numbers by a microscopic amount. This operation repeats across every sentence in the training corpus; every token that participates in a forward pass has its embedding vector nudged by the resulting gradient. A corpus of billions of words means billions of gradient updates, each one a tiny push in the embedding space. This chapter traces the cumulative effect of those pushes—and the geometric structure that organically emerges from them.

From One Nudge to a Million

In Chapters 4 and 5, the gradient was derived for a single training example: the sequence `The` `quick` `brown` predicting `fox`. That gradient adjusted exactly one row of W_E —the row for `brown`—in a direction that would make the network slightly more likely to predict `fox` in this context.

But `brown` does not appear only before `fox`. Across a large training corpus, `brown` precedes thousands of different words:

- `brown` `fox` — our toy example
- `brown` `bear` — a different animal
- `brown` `sugar` — a food ingredient
- `brown` `eyes` — a physical description
- `brown` `shoes` — an article of clothing

Each of these contexts runs the same pipeline from Chapter 4: a forward pass produces a prediction, the loss measures the error, and backpropagation computes a gradient $\frac{\partial L}{\partial \mathbf{h}} = (\hat{\mathbf{y}} - \mathbf{y}) \cdot W_U^T$ that nudges `brown`'s vector. But each context produces a *different* gradient, because the target word is different and the error signal $\hat{\mathbf{y}} - \mathbf{y}$ points in a different direction.

The learning rate α ensures that no single gradient dominates. Each individual nudge is microscopic. Over millions of iterations, these nudges accumulate *statistically*: gradient components that are consistently reinforced across many different contexts grow dominant, while components that point in contradictory directions cancel out. The final position of `brown`'s vector is not determined by any single sentence—it is the aggregate result of every context in which `brown` ever appeared.

This is a mathematical realization of the distributional hypothesis introduced in Chapter 3. Chapters 3, 4, and 5 demonstrate the precise mechanism by which a neural network discovers this principle on its own: gradient descent, applied at scale, forces words with similar contextual distributions into similar regions of the embedding space. No explicit notion of "meaning" is ever provided to the network. Meaning emerges as a geometric side effect of learning to predict the next word.

Convergent Geometry

The distributional hypothesis does not only explain why a single word's vector stabilizes—it explains why *different* words end up near each other.

Consider two words from our toy vocabulary: `fox` and `dog`. Both are animals. In a large corpus, when they appear as the input token, they frequently predict the same next words:

- `The` **`fox`** `jumps over the fence` / `The` **`dog`** `jumps over the fence`
- `A wild` **`fox`** `appeared nearby` / `A wild` **`dog`** `appeared nearby`
- `The` **`fox`** `was spotted nearby` / `The` **`dog`** `was spotted nearby`

This overlap has a direct mathematical consequence. To see it precisely, the gradient formula from Chapters 4 and 5 is traced for two specific training examples, both using `jumps` (index 4 in our toy vocabulary) as the shared target.

Example 1: `fox` as input, predicting `jumps`. The network runs the forward pass from Chapter 3—extracting `fox`'s embedding from W_E, projecting through W_U, and applying Softmax—to produce a predicted probability distribution. Suppose the network outputs:

$$\hat{\mathbf{y}}_{fox} = [0.05 \quad 0.05 \quad 0.05 \quad 0.10 \quad 0.30 \quad 0.20 \quad 0.10 \quad 0.15]$$

The network assigns 30% probability to `jumps` at index 4. The ground truth $\mathbf{y}$ is a one-hot vector with a 1 at index 4 and 0 everywhere else. Computing the error signal $\hat{\mathbf{y}} - \mathbf{y}$ from Chapter 4 element by element:

$$\hat{\mathbf{y}}_{fox} - \mathbf{y} = [0.05 \quad 0.05 \quad 0.05 \quad 0.10 \quad \mathbf{-0.70} \quad 0.20 \quad 0.10 \quad 0.15]$$

Seven of the eight components are small positive numbers (the largest is 0.20 for `over`). The eighth—at index 4, the correct answer `jumps`—is $0.30 - 1 = -0.70$: a large negative value whose magnitude is more than three times larger than any other component.

Example 2: `dog` as input, also predicting `jumps`. In a different sentence, `dog` is the input token. Because `dog`'s embedding is different from `fox`'s, the forward pass produces a different prediction:

$$\hat{\mathbf{y}}_{dog} = [0.08 \quad 0.03 \quad 0.07 \quad 0.12 \quad 0.25 \quad 0.15 \quad 0.12 \quad 0.18]$$

But the target is the same word `jumps`, so $\mathbf{y}$ is the same one-hot vector:

$$\hat{\mathbf{y}}_{dog} - \mathbf{y} = [0.08 \quad 0.03 \quad 0.07 \quad 0.12 \quad \mathbf{-0.75} \quad 0.15 \quad 0.12 \quad 0.18]$$

The individual predictions differ, but the structure of the error signal is the same: seven small positive values, and one large negative value at index 4. The dominant component sits at the same index in both cases, because both examples share the same target word.

From error signal to gradient direction. The gradient that updates each input token's embedding is $(\hat{\mathbf{y}} - \mathbf{y}) \cdot W_U^T$. Recall that W_U^T is an 8×3 matrix (the transpose of our 3×8 un-embedding matrix). This matrix multiply computes a weighted sum of the rows of W_U^T, where the eight components of $\hat{\mathbf{y}} - \mathbf{y}$ serve as the weights. Let $\mathbf{r}_j$ denote row j of W_U^T—a 3-dimensional vector representing vocabulary word j's un-embedding weights. Expanding the `fox` example:

$$\frac{\partial L}{\partial \mathbf{h}_{fox}} = 0.05 \cdot \mathbf{r}_0 + 0.05 \cdot \mathbf{r}_1 + 0.05 \cdot \mathbf{r}_2 + 0.10 \cdot \mathbf{r}_3 - 0.70 \cdot \mathbf{r}_4 + 0.20 \cdot \mathbf{r}_5 + 0.10 \cdot \mathbf{r}_6 + 0.15 \cdot \mathbf{r}_7$$

This is a sum of eight 3-dimensional vectors, each scaled by its coefficient. The term $-0.70 \cdot \mathbf{r}_4$ dominates the sum: its coefficient has the largest magnitude by a factor of more than three, and it is the only negative term. The gradient direction is therefore determined primarily by $\mathbf{r}_4$—the un-embedding weights for `jumps`.

For the `dog` example, the same expansion gives:

$$\frac{\partial L}{\partial \mathbf{h}_{dog}} = 0.08 \cdot \mathbf{r}_0 + 0.03 \cdot \mathbf{r}_1 + 0.07 \cdot \mathbf{r}_2 + 0.12 \cdot \mathbf{r}_3 - 0.75 \cdot \mathbf{r}_4 + 0.15 \cdot \mathbf{r}_5 + 0.12 \cdot \mathbf{r}_6 + 0.18 \cdot \mathbf{r}_7$$

The same row $\mathbf{r}_4$ dominates, with a similarly large negative coefficient (-0.75 vs. -0.70). The small positive contributions differ between the two examples, but the dominant direction—determined by the shared target `jumps` through $\mathbf{r}_4$—is the same. Both gradients push both embeddings in approximately the same direction.

The words `fox` and `dog` share *many* targets across the corpus: `jumps`, `runs`, `sleeps`, `was`, and countless others. Each shared target places the dominant component of the error signal at the same index, causing both gradients to be pulled through the same row of W_U^T. Over millions of training examples, these consistent pushes accumulate, pulling `fox`'s and `dog`'s vectors toward the same region of the embedding space. Meanwhile, words that rarely share targets—like `fox` and `over`—receive error signals whose dominant components sit at different indices, producing dissimilar gradients that keep their vectors distant.

The result is **semantic clustering**: after training, the embedding space is no longer the featureless, isotropic expanse described in Chapter 2. It has acquired structure. Animals cluster near other animals. Verbs cluster near other verbs. Adjectives describing color cluster near other color adjectives. None of this structure was programmed into the network—it emerged organically from the statistics of the training data, mediated entirely by the gradient descent process derived in Chapters 4 and 5.

Measuring the Geometry

To make precise claims like "`fox` and `dog` are close in the embedding space," the geometric relationship between two vectors must be quantified. The standard metric is **cosine similarity**, which measures the cosine of the angle between two vectors in d-dimensional space.

The Formula

Given two vectors $\mathbf{a}$ and $\mathbf{b}$, each with d dimensions, their **dot product** is first defined as the element-wise product of corresponding dimensions, summed into a single scalar:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{m=1}^{d} a_m \cdot b_m$$

Each vector's **magnitude** (its Euclidean norm) is also required—the length of the vector in d-dimensional space:

$$\|\mathbf{a}\| = \sqrt{\sum_{m=1}^{d} a_m^2}$$

Cosine similarity divides the dot product by the product of the two magnitudes. This normalization removes the effect of vector length, isolating the purely directional relationship between the two vectors:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

The result is a single scalar between -1 and $+1$:

- $+1$: The vectors point in exactly the same direction. Maximum similarity.
- 0: The vectors are perpendicular (orthogonal). No directional relationship.
- -1: The vectors point in exactly opposite directions. Maximum dissimilarity.

A Concrete Computation

To ground this formula in our toy model, consider two hypothetical post-training vectors. After billions of gradient updates, suppose the 3-dimensional vectors for `fox` and `dog` have settled at:

$$\mathbf{v}_{fox} = [0.9 \quad -0.4 \quad 0.3], \quad \mathbf{v}_{dog} = [0.7 \quad -0.1 \quad 0.5]$$

Dot product — multiply each pair of corresponding dimensions and sum:

$$\mathbf{v}_{fox} \cdot \mathbf{v}_{dog} = (0.9)(0.7) + (-0.4)(-0.1) + (0.3)(0.5) = 0.63 + 0.04 + 0.15 = 0.82$$

Magnitudes — square each dimension, sum, and take the square root:

$$\|\mathbf{v}_{fox}\| = \sqrt{0.9^2 + (-0.4)^2 + 0.3^2} = \sqrt{0.81 + 0.16 + 0.09} = \sqrt{1.06} \approx 1.030$$

$$\|\mathbf{v}_{dog}\| = \sqrt{0.7^2 + (-0.1)^2 + 0.5^2} = \sqrt{0.49 + 0.01 + 0.25} = \sqrt{0.75} \approx 0.866$$

Cosine similarity — divide the dot product by the product of the magnitudes:

$$\cos(\mathbf{v}_{fox}, \mathbf{v}_{dog}) = \frac{0.82}{1.030 \times 0.866} = \frac{0.82}{0.892} \approx 0.92$$

A cosine similarity of 0.92 —close to the maximum of $+1.0$ —reflects the fact that both words are animals that appear in highly overlapping contexts. This contrasts with the result from Chapter 2: at initialization, randomly initialized vectors in high-dimensional space are mutually orthogonal, with cosine similarities near 0.0 . Training transforms the geometry into a space where semantic relationships are encoded as directional proximity.

Linear Substructures

Beyond semantic clustering, the emergent geometry of trained embeddings exhibits **linear substructures**, where simple vector arithmetic captures semantic and syntactic relationships.

The most famous example involves the words `king`, `queen`, `man`, and `woman`. These four words are not in the 8-word toy vocabulary, but the phenomenon they illustrate emerges in any embedding space trained on a sufficiently large corpus. After training, the following vector arithmetic holds to a close approximation:

$$\mathbf{v}_{king} - \mathbf{v}_{man} + \mathbf{v}_{woman} \approx \mathbf{v}_{queen}$$

The difference $\mathbf{v}_{king} - \mathbf{v}_{man}$ is a **displacement vector**—an arrow pointing from `man`'s position to `king`'s position in the embedding space. This displacement captures everything that distinguishes `king` from `man` while discarding everything they share. What remains is, approximately, the concept of royalty—encoded as a direction and magnitude in the vector space. Adding this same displacement to $\mathbf{v}_{woman}$ translates `woman`'s position along the "royalty" direction, arriving at a point that combines "woman" with "royalty." That point is, approximately, where `queen` sits in the embedding space.

This is not an isolated curiosity. Trained embedding spaces exhibit consistent linear substructures along many different axes:

- Gender: $\mathbf{v}_{brother} - \mathbf{v}_{sister} \approx \mathbf{v}_{king} - \mathbf{v}_{queen}$
- Tense: $\mathbf{v}_{walking} - \mathbf{v}_{walked} \approx \mathbf{v}_{swimming} - \mathbf{v}_{swam}$
- Geography: $\mathbf{v}_{Paris} - \mathbf{v}_{France} \approx \mathbf{v}_{Tokyo} - \mathbf{v}_{Japan}$

None of these relationships were explicitly programmed. No loss function asked the network to encode gender as a linear direction, or verb tense as a consistent vector offset. They emerged purely from the distributional statistics of the training corpus—the same gradient descent process from Chapters 4 and 5, operating on the same $(\hat{\mathbf{y}} - \mathbf{y})$ error signal, applied billions of times. The training objective was simply next-token prediction. The geometric structure is a side effect.

Chapter 7: The Embedding Tensor and the Limits of Static Representations

The previous chapter established the convergent geometry and linear substructures that emerge from gradient descent at scale. This chapter examines how these embeddings operate when processing full sequences rather than individual tokens.

Throughout Chapters 1 through 4, a single token was processed at a time. The input was one one-hot vector $\mathbf{x} \in \mathbb{R}^{1\times 8}$, and the output was one embedding vector $\mathbf{h} \in \mathbb{R}^{1\times 3}$—a single row extracted from W_E. But real language models do not process one word in isolation. They ingest entire sequences. The generalization from single-token lookup to full-sequence processing is natural—and it is the final mechanical step before handing off to the Transformer architecture.

Instead of feeding a single one-hot vector into W_E, T one-hot vectors—one per token in the input sequence—are stacked into a matrix. The variable T denotes the **sequence length**: the number of tokens being processed simultaneously.

Consider our full toy sentence: `The` `quick` `brown` `fox`. This is a sequence of $T = 4$ tokens. Each token has a one-hot representation in $\mathbb{R}^{1\times 8}$ (one element per vocabulary word). Each one-hot vector is constructed individually— `The` places a 1 at index 0, `quick` at index 1, `brown` at index 2, and `fox` at index 3—then all four are stacked vertically into the **one-hot input matrix**:

$$X_{\text{one-hot}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}_{4\times 8}$$

The first row is the one-hot encoding for `The` (a 1 at index 0, zeros elsewhere). The second row is `quick` (a 1 at index 1). The third row is `brown` (index 2), and the fourth row is `fox` (index 3). The shape of this matrix is $T \times V = 4 \times 8$: one row per token in the sequence, one column per word in the vocabulary.

This matrix is then multiplied by the same $W_E \in \mathbb{R}^{8\times 3}$ from Chapter 3:

$$X = X_{\text{one-hot}} \times W_E$$

As established in Chapter 1, multiplying a single one-hot vector by W_E extracts the corresponding row—the dot product with a one-hot vector zeroes out every row except the one where the 1 appears. The same mechanism applies to each row of $X_{\text{one-hot}}$ independently. The matrix multiply performs all four lookups simultaneously, extracting four rows from W_E in a single operation:

$$X = \begin{bmatrix} 0.1 & -0.4 & 0.2 \\ 0.5 & 0.1 & -0.8 \\ -0.3 & 0.9 & 0.4 \\ 0.2 & -0.2 & 0.1 \end{bmatrix}_{4\times 3}$$

Each row of X is the embedding vector for one token in the sequence:

- Row 0: $[0.1 \quad -0.4 \quad 0.2]$ — the embedding for `The`
- Row 1: $[0.5 \quad 0.1 \quad -0.8]$ — the embedding for `quick`
- Row 2: $[-0.3 \quad 0.9 \quad 0.4]$ — the embedding for `brown`
- Row 3: $[0.2 \quad -0.2 \quad 0.1]$ — the embedding for `fox`

Row 2 is precisely the vector $\mathbf{h} = [-0.3 \quad 0.9 \quad 0.4]$ used throughout Chapters 3 and 4 as the hidden state for `brown`. The sequence operation has not changed the lookup mechanism—it has simply batched it.

The result $X \in \mathbb{R}^{T \times d_{model}}$ is the **embedding tensor** for the input sequence: a matrix of T rows and d_{model} columns, encoding the entire sequence as a block of continuous vectors. In our toy model, this is a 4×3 matrix. In a production model processing a 512-token sequence with $d_{model} = 512$, this would be a 512×512 matrix—each of its 512 rows a rich, high-dimensional vector whose position has been shaped by billions of gradient updates into a point that encodes the distributional signature of its token.

The Limits of Static Geometry

The embedding tensor X is dense, continuous, and semantically structured. Each row carries the distributional signature of its token, distilled from billions of training contexts into d_{model} dimensions. But this representation has two fundamental limitations that no amount of additional training can overcome—because they are structural consequences of using a static, context-independent embedding matrix.

Context-blindness. Every occurrence of a word looks up the *same* row of W_E, regardless of the surrounding sentence. The word `apple` in "I ate a delicious `apple`" and `apple` in "I bought an `apple` laptop" produce identical embedding vectors. The trained vector for `apple` is a compromise—the geometric center-of-mass of all its usages, capturing some blend of "fruit" and "technology" but faithfully representing neither. This is the **superposition** problem identified at the end of Chapter 3: a single static vector cannot represent multiple distinct meanings.

Order-agnosticism. The matrix multiplication $X_{\text{one-hot}} \times W_E$ treats each row of $X_{\text{one-hot}}$ independently. It does not know or care which row comes first. The sequences `dog bites man` and `man bites dog`—which carry very different meanings—produce the *same set* of embedding vectors, merely arranged in different rows of X. The embedding matrix has no mechanism to encode word order, because each row's lookup depends only on which vocabulary index contains the 1—not on the row's position within the sequence.

These limitations are not flaws in the training process. They are inherent to the architecture: a fixed lookup table that maps each word to a single, pre-computed vector cannot incorporate the dynamic context of a specific sentence. Resolving context-dependence and encoding positional information requires a fundamentally different mechanism—one that can examine the *entire* sequence of embedding vectors and dynamically re-weight each one based on its neighbors.

That mechanism is the subject of an entirely different architecture. The embedding tensor $X \in \mathbb{R}^{T \times d_{model}}$—with all its semantic richness and all its limitations—is exactly what enters the **Transformer**. There, positional encodings inject word-order information that the embedding lookup cannot provide, and the self-attention mechanism allows each vector in the sequence to attend to every other vector, dynamically resolving ambiguity and context-dependence on a per-sentence basis. The static geometry built across this series is not the final representation—it is the starting point from which the Transformer begins its work.